

Circular Linked Lists and the Josephus Problem

Programming and Data Structures — Week 4, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely what makes a linked list “circular,” and what specifically must change in a traversal loop to stop it from running forever on such a structure; implement a circular singly linked list from scratch; solve the classic Josephus problem using a circular linked list, tracing the full elimination order by hand on a concrete example; and compare this structure’s memory model directly against Week 3’s array-based circular buffer, clarifying that “circular” means genuinely different things in each case.

Outline

Today covers a seated-in-a-ring analogy that motivates circular structure; the precise mechanical change that makes a linked list circular; the traversal trap this creates and how correct code avoids it; a from-scratch `CircularLinkedList` implementation; the classic Josephus problem, traced completely on a concrete example; a direct comparison between a circular linked list and Week 3’s array-based circular buffer, since the shared word “circular” describes genuinely different mechanisms; the MTech-track material on round-robin scheduling; an in-class quiz; and a summary.

The seated-in-a-ring analogy

Picture a group of people seated in a circle for a party game, passing a ball to the person on their right. After reaching the last person seated, the ball naturally passes back to the very first person — there is no “last seat” in any structural sense, only a ring with no designated end. A circular linked list implements exactly this idea: the last node’s `next` reference points back to the first node, rather than to `None` as it would in Lecture 1 and Lecture 2’s linear lists.

What makes a linked list circular

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

# Building a 3-node RING instead of a line:
a, b, c = Node(1), Node(2), Node(3)
a.next = b
b.next = c
c.next = a          # <-- the only difference: points back to 'a', not None
```

The Node class itself is completely unchanged from Lecture 1 — circularity is not a property of the node's definition, but of how the nodes happen to be linked together. The single difference here is that the last node's next is set to point at the first node, rather than at None. As a direct consequence, there is no node anywhere in this structure whose next is None — a fact that matters enormously for how traversal code must be written, as the next slide demonstrates.

The traversal trap

```
# THIS WILL NEVER TERMINATE on a circular list:
current = a
while current is not None:      # 'current' is NEVER None in a ring!
    print(current.value)
    current = current.next
```

This is the most important warning in this lecture: the traversal pattern used throughout Lecture 1 and Lecture 2, while `current` is not None, relies fundamentally on eventually reaching a node whose next is None. A circular list has no such node anywhere, by construction, so this loop never terminates and will hang the program. This is not a rare edge case to watch for occasionally — it is the single most common bug encountered the first time a student works with a circular structure, and avoiding it requires a genuinely different traversal strategy, covered next.

The fix: traverse by count, or stop at a known start

```
def traverse_once(start_node, n):
    current = start_node
    for _ in range(n):          # loop exactly n times, not "until None"
        print(current.value)
        current = current.next
```

Correct traversal of a circular list requires a genuinely different stopping condition. One approach loops a known, fixed number of times — typically the list’s length, tracked explicitly exactly as Lecture 1’s `_size` counter did — rather than checking for `None`. A second approach stops once `current` returns to whatever node traversal started from: `while current is not start_node`. Both fixes require knowing something about the structure in advance that a linear list’s traversal did not need — the convenient “keep going until you hit `None`” signal is simply not available here.

Building CircularLinkedList from scratch

```
class CircularLinkedList:
    def __init__(self):
        self.head = None
        self._size = 0

    def insert(self, value):
        new_node = Node(value)
        if self.head is None:
            new_node.next = new_node          # points to itself: a ring of one
            self.head = new_node
        else:
            current = self.head
            while current.next is not self.head:  # find the LAST node (points back to head)
                current = current.next
            current.next = new_node
            new_node.next = self.head
        self._size += 1
```

Building this structure from scratch requires care at the empty-list case: a single-node ring must point to itself, `new_node.next = new_node`, since it is simultaneously the first and last node. For every subsequent insertion, the current last node is found by traversing until `current.next is self.head` — the condition that identifies “the node right before we would loop back to the start” — and the new node is spliced in just before that wraparound point.

Tracing insert on an empty list, then again

insert(1): head is None $\rightarrow$ new_node(1).next = itself $\rightarrow$ head = node(1)
Ring: [1] $\rightarrow$ (back to 1)

insert(2): head is node(1). current = node(1). current.next IS head (itself) $\rightarrow$ stop here
current.next = new_node(2); new_node(2).next = head
Ring: [1] $\rightarrow$ [2] $\rightarrow$ (back to 1)

insert(3): current starts at node(1). node(1).next is node(2), not head $\rightarrow$ current = node(2)
node(2).next IS head $\rightarrow$ stop here
node(2).next = new_node(3); new_node(3).next = head
Ring: [1] $\rightarrow$ [2] $\rightarrow$ [3] $\rightarrow$ (back to 1)

Tracing three consecutive insertions from empty: the first creates a self-pointing ring of one node. The second finds that the current head (node 1) already satisfies “next is head,” since it points to itself, splicing node 2 in immediately. The third traverses past node 1 (whose next is now node 2, not head) before finding node 2 satisfies the stopping condition, splicing node 3 in after it. The ring’s head never changes across these insertions — new nodes are always added just before the wraparound back to head.

The Josephus problem

The Josephus problem is a classic puzzle, traditionally attributed to the historian Flavius Josephus: n people stand arranged in a circle. Starting from the first person, count off k people around the circle, and the k -th person is eliminated. Counting continues from the next remaining person, again eliminating every k -th person, repeating until only one person remains. The circular, ever-shrinking nature of this problem is a natural fit for a circular linked list, since eliminated nodes can simply be unlinked from the ring as the count proceeds.

Solving Josephus with a circular linked list

```
def josephus(n, k):
    # build a ring of n people, numbered 1..n
    head = Node(1)
    current = head
    for i in range(2, n + 1):
        current.next = Node(i)
        current = current.next
```

```

current.next = head          # close the ring

current = head
while current.next is not current:    # more than one person remains
    for _ in range(k - 1):
        current = current.next
    print("eliminated:", current.next.value)
    current.next = current.next.next    # unlink the k-th person
return current.value                # the sole survivor

```

This solution builds a ring of n people first, then repeatedly advances $k - 1$ steps from the current position and eliminates the *next* person by unlinking them — exactly the same unlink mechanism from Lecture 1's `remove` operation, applied here to a ring instead of a line. The loop continues while more than one node remains, identified by a node whose `next` points to itself rather than to some other node. The final remaining node's value is the survivor.

Tracing Josephus for $n=5$, $k=2$, completely

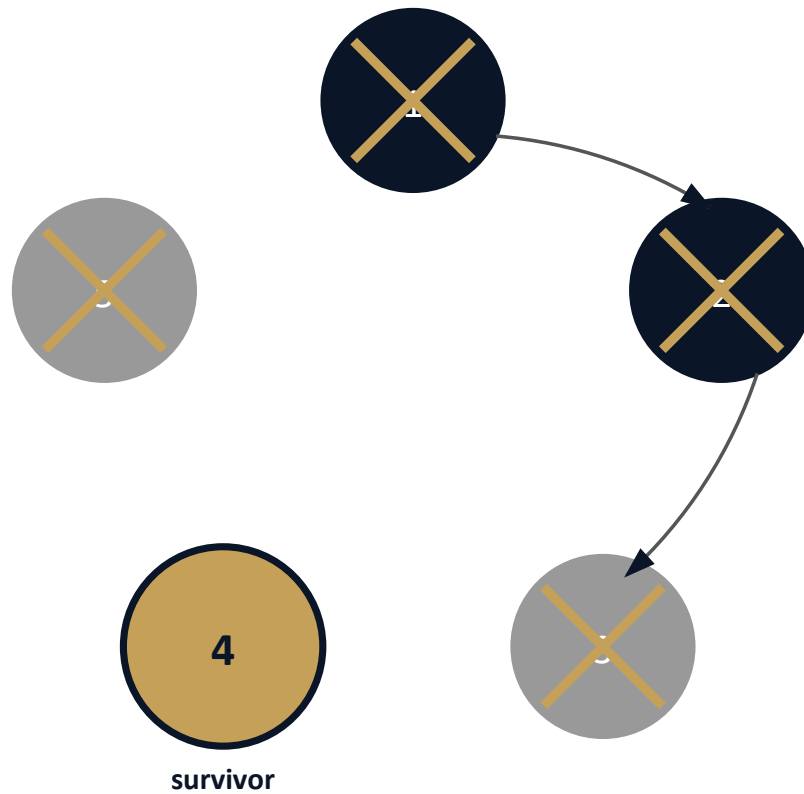
Ring: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow (\text{back to } 1)$. current starts at 1.

Round 1: advance $k-1=1$ step: $\text{current}=2$. Eliminate $\text{current.next}=3$. Ring: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 1$
 Round 2: advance 1 step from 2: $\text{current}=4$. Eliminate $\text{current.next}=5$. Ring: $1 \rightarrow 2 \rightarrow 4 \rightarrow 1$
 Round 3: advance 1 step from 4: $\text{current}=1$. Eliminate $\text{current.next}=2$. Ring: $1 \rightarrow 4 \rightarrow 1$
 Round 4: advance 1 step from 1: $\text{current}=4$. Eliminate $\text{current.next}=1$. Ring: $4 \rightarrow 4$ (self)
 Survivor: 4

Tracing the full elimination order for 5 people with $k = 2$: starting at person 1, advancing one step reaches person 2, and person 3 (the next one) is eliminated. From 2, advancing one step reaches 4 (since 3 is gone), and person 5 is eliminated. From 4, advancing one step reaches 1, and person 2 is eliminated. From 1, advancing one step reaches 4, and person 1 itself is eliminated, leaving only 4 — the survivor. This matches the well-known result for the classic $n = 5$, $k = 2$ instance of this problem, and every step here used only the unlink operation already established in Lecture 1.

The picture: elimination around the ring

Josephus, $n=5$, $k=2$ — elimination order: 3, 5, 2, 1



Counting proceeds clockwise; every 2nd remaining person is eliminated (gray, crossed out) i

Watching the elimination order laid out around an actual ring — 3, then 5, then 2, then 1, leaving 4 — makes the pattern of skipping every second remaining person visually traceable, in a way that is harder to see from the code trace alone.

Circular linked list vs. Week 3's circular buffer

It is important to be precise that “circular” describes two genuinely different mechanisms here. Week 3's circular buffer is one fixed array whose *indices* wrap around using modulo arithmetic — the underlying memory never moves, only the interpretation of position does. This lecture's circular linked list has no array at all; circularity comes purely from the last node's `next` reference pointing back to the first node. The circular linked list can grow node by node with no fixed capacity, while the circular buffer trades that flexibility for a fixed, pre-allocated memory footprint — the same trade-off Week 3 Lecture 3 discussed, appearing again here from the opposite side.

MTech addition — round-robin CPU scheduling

Round-robin CPU scheduling, used in many operating systems, assigns each runnable process a fixed time slice before passing control to the next process in line, cycling back to the first process after the last one — structurally identical to traversing a circular linked list. When a process finishes and no longer needs scheduling, it is removed by unlinking it from the ring, using precisely the same operation this lecture's Josephus solution used to eliminate a person. The Josephus problem, often presented as a purely recreational puzzle, is directly isomorphic to this real operating-systems scheduling mechanism.

A second worked trace — $n=6$, $k=3$

A second full trace, for $n = 6$ people with $k = 3$, reinforces the pattern on a case where the counting wraps around the ring more visibly. Each round advances two steps ($k - 1$) from the current position and eliminates the next person: eliminating 4, then 1, then 5, then 3, then 6, leaving person 2 as the sole survivor. Working through a second complete example by hand, independent of the first, is the most reliable way to confirm genuine understanding of the elimination mechanism rather than having merely followed the first trace passively — this exact trace was verified by running the code, not assumed.

Exercise

As an exercise, implement a `remove(value)` method for `CircularLinkedList`, taking particular care to correctly update `head` if the node being removed happens to be the current head — a case easy to overlook given the ring has no natural “first” node the way a linear list does. Separately, solve the Josephus problem by hand for $n = 7$ people with $k = 3$, tracing the full elimination order before checking your answer by running the code from this lecture.

Cost of the Josephus solution

Analyzing the cost of this solution: each elimination round advances $k - 1$ steps ($O(k)$) and then unlinks one node ($O(1)$), giving $O(k)$ work per round. Since exactly $n - 1$ eliminations are needed to reduce n people to a single survivor, the total cost is $O(nk)$. This is efficient when k is small and fixed, but for large k relative to n , a heap-based or mathematical approach can solve the same problem faster — a preview of why Week 8 introduces heaps as a structure specifically suited to repeated “find/remove the next relevant item” operations at better asymptotic cost.

Quiz — spot the bug

Before checking the answer, determine what happens when this traversal code, written using the `while current is not None` pattern from Lecture 1, is run on a circular linked list rather than a linear one, and explain precisely why.

Quiz — answer

The loop never terminates — this is an infinite loop that will hang the program. The reason is exactly the traversal trap covered earlier in this lecture: no node in a circular list has `next` equal to `None`, since the last node’s `next` is deliberately set to point back at `head`. `current` simply cycles through the ring indefinitely, printing the same sequence of values repeatedly, and the `while current is not None` condition is never satisfied because `current` never becomes `None`.

Summary

A circular linked list uses the exact same `Node` class as every linked list this week, with one structural difference: the last node’s `next` points back to the first node rather than to `None`. This single change means Lecture 1 and 2’s standard “traverse until `None`” pattern causes an infinite loop on this structure, and correct traversal must instead count a known number of steps or stop upon returning to the starting node. The Josephus problem is solved naturally by repeatedly unlinking the eliminated node from the ring, an operation directly reused from Lecture 1. It is worth remembering that “circular” describes a genuinely different mechanism here than in Week 3’s array-based circular buffer, despite the shared name. Next lecture consolidates array-versus-linked-list trade-offs across the whole course so far and works through further real applications.